

Controllable Labels

Labels the controller is allowed to enable/disable:

```

automaton SpiMaster {
  controllable {
    label cs_assert;
    label cs_deassert;
    label sclk_tick;
    label mosi_write;
    label done;
  }
  states { ... }
  transitions { ... }
}

```

Declared *inside* the automaton, before states.

Uncontrollable Labels

Any label **not** listed in the `controllable` block is *uncontrollable* — it represents an environment event the controller cannot prevent.

The controller must tolerate all uncontrollable transitions: it can only restrict which controllable labels fire.

Design Guideline

Start by making all labels controllable. Move labels to uncontrollable only when they represent genuine environment actions (sensor inputs, external interrupts, etc.).

Controller Declaration

A controller restricts an automaton to satisfy a formula:

```

controllers {
  controller spi_safety {
    source SpiMaster;
    satisfying safety_invariant;
  }
}

```

- **source** — the automaton (or composition) to control
- **satisfying** — the mu-calculus formula to enforce

Labeled Modalities

Target specific labels in formulas:

Syntax	Meaning
$\langle \text{labels}=\{\mathbf{a}\} \rangle \varphi$	<i>Diamond</i> : $\exists$ an $\mathbf{a}$ -successor satisfying φ
$[\text{labels}=\{\mathbf{a}\}] \varphi$	<i>Box</i> : $\forall$ $\mathbf{a}$ -successors satisfy φ

Example — from Idle, `cs_assert` reaches Selected:

```

body = (! Idle) ||
  < labels = { cs_assert } > Selected;

```

CLI Synthesis

```

# Synthesize a controller
mununu context synth file.ctxdsl \
  --formula safety_invariant \
  --automaton SpiMaster

# Evaluate (check without synthesizing)
mununu context eval file.ctxdsl \
  --formula transfer_completes \
  --automaton SpiMaster

```

Synthesis Output

- **Realizable** — a controller exists that enforces the property. Output shows the number of states retained.
- **Unrealizable** — no controller can enforce the property under all environment behaviors.

Composition Rule

Controllable labels must **not** be shared between automata in a composition. If two automata both declare a label as controllable, synthesis results are undefined. Use distinct label names or keep shared labels uncontrollable.

Useful Formula Patterns

```

// Can always return to Idle
mu X. (Idle || <> X)

// All cs_assert transitions safe
[labels = { cs_assert }] Safe

// Liveness under control
nu X. (mu Y. (Goal || <> Y)) && ([ ] X)

```